PPT CONTENT — Dual-Mode Vision-Guided Robotic Arm

> Based on engineering project PPT template | ~20 Slides

Slide 1 — Title Slide

Design and Development of a Dual-Mode Vision-Guided Robotic Arm
for Autonomous Sorting & Human-in-the-Loop Manipulation

Submitted by:

- [Your Name] — [Reg No]

Under the Guidance of:

- [Guide Name]

Department of Electronics and Communication Engineering
[College / University Name]

Academic Year: 2025 – 2026

Slide 2 — Abstract

- Industrial robotic sorting systems exist — but they are **rigid, single-mode, and cannot adapt** when a human needs to intervene safely and immediately
- This project designs, simulates, and implements a **6-DOF robotic arm** that autonomously perceives, classifies, and sorts objects using a full **AI-powered computer vision pipeline** (YOLOv8 + OpenCV + camera calibration)
- On top of the autonomous sorting system, the project solves a distinct open problem in robotics: **seamless human-in-the-loop takeover** — the human raises a hand, the robot yields control; the hand leaves, the robot resumes — zero UI interaction
- The human override extends globally: **any person, anywhere in the world**, can take control via a phone camera and the internet (MQTT teleoperation) with ~250 ms round-trip latency
- Entire system is built on **open-source industrial-grade tools** (ROS2 Jazzy, Gazebo Harmonic, MoveIt2) with a custom hardware interface via **ESP8266 + PCA9685**
- **Expected outcome:** A research-grade autonomous sorting robot with a fully integrated, interrupt-safe, context-aware human control layer — deployable on low-cost hardware and validated in simulation before physical deployment

---

Slide 3 — Introduction

The Scale of the Problem

PROF

- The global industrial robotics market is valued at **$55 billion (2024)** — the majority of deployed systems perform **fixed, pre-programmed tasks with no perception**
- A typical sorting line stops entirely when a human needs to intervene — costing time, money, and creating safety risks during the transition
- The **Collaborative Robot (Cobot) market** is the fastest-growing segment ($12B+ by 2030) precisely because industries need robots that work *alongside* humans, not just instead of them

What This Project Addresses

- **Perception gap:** Most low-cost arms are blind — they follow fixed coordinates. This project builds a full **AI vision pipeline** so the arm understands what it is looking at
- **Autonomy gap:** Most research arms either operate fully autonomously or are fully teleoperated. This project implements **both, in a single unified system**
- **Override gap:** There is no standard, low-cost mechanism for a human to interrupt an autonomous robot safely and intuitively — this project solves it with **gesture-based takeover**
- **Access gap:** Remote teleoperation requires expensive proprietary hardware. This project demonstrates it works with a **phone camera and free internet infrastructure**

Motivation

- Build a **complete robotic pipeline** — perception → planning → execution — not just a demo
- Validate that **open-source tools (ROS2, MoveIt2, YOLOv8, MediaPipe)** are sufficient for industrial-class capability
- Produce a reproducible platform for **Human-Robot Interaction research** on a ₹15,000 hardware budget

---

Slide 4 — Problem Statement

Three Distinct Engineering Problems Being Solved

Problem 1 — Autonomous Perception & Sorting

PROF

- A robot arm must *see* the world, classify what it sees, compute 3D coordinates from a 2D camera image, plan a collision-free path, and physically pick and place an object — entirely without human input
- This alone is a complete robotics research problem: **perception** → **localisation** → **planning** → **execution**

Problem 2 — Safe, Intuitive Human Override

- When a human needs to intervene in an autonomous system, the intervention must be: **instant** (no lag), **intuitive** (no training required), and **safe** (no collision risk during transition)
- Existing solutions require teach pendants, emergency stops, or software GUIs — none of which are hands-free or context-aware

Problem 3 — Remote Teleoperation on Zero Infrastructure

- Industrial remote arm control systems cost tens of thousands of dollars and require dedicated VPNs, proprietary protocols, and skilled operators
- This project must achieve the same result using a **phone browser, free cloud infrastructure, and a gesture interface** that anyone can use

Combined Objective

- Design a **single unified system** that solves all three problems simultaneously, validated in full physics simulation, deployable on commodity hardware

---

Slide 5 — System Overview

Core System: Autonomous Sorting (Default State)

The arm operates **autonomously by default** — this is not a background state, it is the primary function:

```
Camera → YOLOv8 Object Detection → Object Classification
       → Camera Calibration → Pixel-to-World Coordinate
       → MoveIt2 IK Planning → Collision-Free Trajectory
       → JointTrajectoryController → Arm picks and sorts
```

This is a **complete perception-to-action pipeline** running continuously.

Innovation Layer: Interrupt-Safe Human Control

Layered on top of the autonomous system is a **context-aware human override mechanism**:

Mode	How It Triggers	What Happens
AUTONOMOUS *(primary)*	Default — no human detected	Full AI vision pipeline runs; arm sorts independently
LOCAL GESTURE *(override)*	Hand appears in webcam	Arm immediately yields; MediaPipe controls all 6 joints
REMOTE GESTURE *(override)*	MQTT stream from phone	Global operator takes control via phone browser, anywhere

PROF

The Key Innovation

> *This is not a robot with gesture control bolted on — it is a fully autonomous sorting robot that also solves the human override problem. Both systems are production-grade and operate within the same unified control architecture.*

Mode switching guarantees:

- Zero UI interaction required
- Current trajectory completes safely before transition

- Autonomous operation resumes automatically when human steps away

---

Slide 6 — Autonomous Sorting Pipeline (Technical Depth)

What the Arm Does Without Any Human

Stage 1 — Object Detection (YOLOv8)

- Live camera feed processed by YOLOv8 running on **NVIDIA RTX 3060 (CUDA)**
- Detects object class (colour, shape, type) and 2D bounding box in real-time
- Inference time: ~15–20 ms per frame at full HD

Stage 2 — 3D Localisation (Camera Calibration)

- Camera intrinsics calibrated using OpenCV checkerboard method
- 2D pixel coordinates → 3D world coordinates using known workspace plane + homography
- Output: `(x, y, z)` in robot base frame

Stage 3 — Motion Planning (MoveIt2)

- Target pose sent to MoveIt2 motion planner
- **Inverse Kinematics (IK)** solver computes joint angles to reach target
- **Collision-free path planning** using OMPL — avoids workspace obstacles
- Trajectory optimised for smoothness and time

Stage 4 — Execution (ros2_control)

- `JointTrajectoryController` executes computed trajectory
- Joint positions tracked at hardware level via `ros2_control` abstraction
- Same interface works for both **Gazebo simulation and real hardware** — no code change

Stage 5 — Return and Loop

- Gripper releases at target bin, arm returns to home, next object cycle begins
- System runs indefinitely — no human input required

PROF

> *This is a full perception-to-execution autonomous robotic pipeline — comparable to what is deployed in industrial pick-and-place installations, implemented on open-source tools.*

7.1 Robotic Arm

Component	Details
Model	6-DOF Metal Servo Arm Kit
Degrees of Freedom	6 (5 arm joints + gripper)
Servo Motors	6× MG996R
Frame	Black anodised aluminium brackets

Component	Details
Gripper	Claw-type, servo-actuated
URDF Source	Adapted from community SolidWorks CAD model

7.2 Electronics

Component	Purpose
ESP8266	Wi-Fi microcontroller — serial bridge to ROS2 *(replaces Arduino Mega for wireless capability)*
PCA9685 (16-channel)	I2C servo driver — controls all 6 servo channels
5V / 10A PSU	Dedicated power supply for servos
Webcam (1080p)	Primary vision input — sorting + local gesture control
Laptop (RTX 3060)	Main compute — runs ROS2, Gazebo, vision pipeline

> **Design Note:** ESP8266 was chosen over Arduino Mega to eliminate the USB tether and enable potential Wi-Fi serial communication in future phases, while maintaining full backward compatibility with the PCA9685 I2C interface.

Slide 8 — Software Stack

Core Infrastructure

Technology	Role
Ubuntu 24.04 LTS	Operating system
ROS2 Jazzy Jalisco	Robot middleware and inter-node communication
Gazebo Harmonic	Physics simulation (official Jazzy pair)
ros2_control	Simulation ↔ Hardware abstraction layer
MoveIt2	Motion planning and inverse kinematics
Python 3.10+	All node logic and vision pipeline

Vision & AI Stack

Library	Purpose
OpenCV 4.8+	Image processing, colour detection, camera calibration
YOLOv8 (Ultralytics)	Object detection & classification (CUDA-accelerated)
MediaPipe 0.10+	21-landmark real-time hand tracking

PROF

Library	Purpose
PyTorch 2.x + CUDA 12.1	GPU acceleration for YOLOv8 on RTX 3060

Remote Control Stack

Technology	Purpose
MQTT (Protocol v5)	Lightweight internet pub/sub messaging
HiveMQ Cloud	Free cloud MQTT broker
Flask + ngrok	Serve phone web app + internet tunnel
MediaPipe.js (Browser)	Hand tracking directly in mobile browser

---

Slide 9 — System Architecture — ROS2 Node Graph

```
/camera_node          → publishes /camera/image_raw
      │
      ├─→ /vision_node         (YOLOv8 + OpenCV — detects & localises
objects)
      │         └─→ /sorting_planner_node  (object → bin → target XYZ)
      │
      └─→ /gesture_node        (MediaPipe — extracts hand landmarks from
webcam)
                └─→ /mode_manager_node     (decides active mode →
/control_mode)

/mqtt_bridge_node     → receives phone commands via internet → ROS2
topics

/mode_manager_node    → monitors all inputs → switches AUTO / LOCAL /
REMOTE

/motion_planner_node  → MoveIt2 wrapper — IK + collision-free path
planning
      └─→ /arm_controller_node (JointTrajectoryController via
ros2_control)
                └─→ /esp8266_bridge    (serial comms → ESP8266 → PCA9685
→ Servos)
```

Central principle: Every data path flows through the mode manager — it is the single authority on what the arm does at any given moment.

Slide 10 — Remote Control Data Flow

PROF

```
PHONE — ANYWHERE IN THE WORLD
  └─ Browser opens web app (via ngrok URL)
  └─ Front camera activates
  └─ MediaPipe.js detects 21 hand landmarks
  └─ Maps landmarks → 6 joint angles (degrees)
  └─ Publishes JSON → HiveMQ MQTT Broker (SSL)
              |
          INTERNET  (~30–80 ms)
              |
  └─ mqtt_bridge_node.py receives JSON
  └─ Converts degrees → radians
  └─ Publishes JointTrajectory → /arm_controller
  └─ MoveIt2 executes → Arm moves
  └─ Feedback sent back to phone (latency display)

  Total end-to-end latency: ~230–290 ms  (fully usable)
```

Why MQTT?

- Designed for unreliable low-bandwidth connections
- Works through firewalls and NAT without port forwarding
- Free cloud broker (HiveMQ) — no server infrastructure needed

---

Slide 11 — URDF & Simulation Development

Challenge: CAD Model → Simulation-Ready URDF

Step	What Was Done
Hardware received	Physical arm arrived — existing URDF files had wrong motor positions
CAD sourcing	Found matching SolidWorks model online from community
Conversion	SolidWorks → **Onshape** (browser CAD) → STL + STEP export via API
URDF obtained	Received exact URDF from community — 40 links, 29 fixed joints, 11 actuated joints
URDF cleanup	**Merged 40 links → 12 links** by computing cumulative transforms using Python/NumPy
Validation	`check_urdf` passes cleanly; all 43 STL meshes correctly placed

Key Simulation Fixes (9 Issues Resolved)

Issue	Fix
Physics Error Code 19 — zero inertia	Replaced zero inertia values with 1e-7 minimum

PROF

Issue	Fix
Robot invisible — mesh URI failed	Set `GZ_SIM_RESOURCE_PATH` in launch file
Unsupported Ogre material strings	Replaced with RGBA `<ambient>/<diffuse>` blocks
Gripper continuously rotating	Changed to `revolute` joints with ±0.5 rad limits
RViz2 drops all `/joint_states`	Created relay node to stamp `frame_id='base_link'`

Result: Clean 12-link robot model running stably in Gazebo Harmonic with full RViz2 TF tree.

Slide 12 — MediaPipe Hand Tracking Integration

What is MediaPipe?

- Google's real-time ML framework for perception tasks
- **Hand Landmarker** detects **21 3D landmarks** per hand at 30+ FPS on CPU
- No GPU required for hand tracking — runs efficiently on laptop webcam feed

Implementation Approach

```
Webcam Frame (1080p)
      |
MediaPipe HandLandmarker
      |
21 Landmarks (x, y, z per point)
      |
Landmark-to-Joint Mapping
  - Wrist position     → Base rotation (Joint 1)
  - Palm orientation   → Shoulder + elbow (Joints 2, 3)
  - Finger curl angles → Wrist + end joints (Joints 4, 5)
  - Thumb–index gap    → Gripper open/close (Joint 6)
      |
JointTrajectory message → /arm_controller
```

Current Status

- ✅ MediaPipe installed and hand detection verified
- 🔄 **Currently working on:** Landmark-to-joint-angle mapping and live arm control in simulation

---

Slide 13 — Development Progress & Roadmap

Completed ✅

Milestone	Details
Hardware sourced	6-DOF arm kit, ESP8266, PCA9685, webcam — all received

Milestone	Details
CAD model converted	SolidWorks → STL via Onshape API — 43 mesh files
Simulation environment	Gazebo Harmonic + RViz2 — robot spawns, TF tree complete
URDF restructured	40-link CAD export → 12-link clean model, all 9 simulation bugs fixed
MediaPipe integrated	Hand detection pipeline functional — 21 landmarks extracted

In Progress 🔄

Milestone	Details
Gesture control → arm	Mapping hand landmarks to 6 joint angles — live simulation control

Next Steps ⏳

Step	Technology
MoveIt2 Setup Assistant	Configure planning groups, SRDF, kinematics solver
Motion planning	IK-based arm control via MoveIt2 interactive markers
Autonomous sorting	YOLOv8 object detection + pick-and-place pipeline
Remote MQTT control	Phone browser → HiveMQ → ROS2 bridge → arm
Hardware interface	ESP8266 firmware + PCA9685 PWM → real servo control

---

Slide 14 — Flowchart — System Operation

```
START
  |
  ▼
System Initialisation
(ROS2 nodes launch, Gazebo spawns robot)
  |
  ▼
Mode Manager — Sense Context
  |
  ├─ MQTT data arriving? ──YES──→ REMOTE GESTURE MODE
  |                                      |
  ├─ Hand in webcam? ──YES──→ LOCAL GESTURE MODE
  |                                      |
  └─ Nothing detected ──────→ AUTONOMOUS MODE
              |                          |
   YOLOv8 detects object         MediaPipe hand tracking
              |                          |
   Sorting planner → target XYZ  Landmark → joint angles
              |                          |
         MoveIt2 IK                  MoveIt2 IK
```

PROF

```
                |                          |
     JointTrajectory → Controller   JointTrajectory → Controller
                |                          |
           Arm moves                  Arm moves
                |
    Mode changes? ──YES──→ Complete current motion → Switch
                |
               LOOP
```

Slide 15 — Safety & Reliability Features

Safety Feature	How It Works
Remote Timeout	No MQTT command for 3 sec → arm returns to home → AUTO mode resumes
Angle Clamping	All joint angles hard-clamped to URDF limits — phone cannot send damaging values
Smoothing Filter	Exponential moving average on gesture input — eliminates hand jitter
Trajectory Duration Floor	150 ms minimum per command — prevents instantaneous joint jumps
QoS 0 — No Queue	Old commands dropped immediately — no stale command backlog
Safe Mode Switch	Mode manager waits for active motion to finish before switching
Home Position Recovery	Any error, disconnect, or timeout → arm returns to predefined safe pose
Joint Limits in URDF	Physical limits enforced at URDF level — MoveIt2 respects them in all planning

PROF

> *These features ensure safe operation whether the arm is running autonomously, under local gesture control, or being operated remotely from an unknown network.*

Slide 16 — Real-World Applications

Manufacturing & Quality Control

- Defect sorting — vision detects faulty products, arm removes them from the production line
- PCB component sorting — classify resistors, ICs, capacitors by visual markings

Healthcare & Assistive Technology

- Assistive robotic arm — gesture control enables mobility-impaired patients to pick up objects
- Lab sample sorting — sort blood vials and specimens by colour-coded labels

Agriculture

- Fruit grading — sort produce by size, colour, and ripeness
- Harvest assistance — gesture-controlled picking for delicate crops

Logistics & E-commerce

- Package sorting — classify and route parcels automatically
- Gesture override for ambiguous items during returns processing

Hazardous Environments

- Hazardous material handling — remote gesture control keeps humans safe
- Low-cost prototype of military teleoperation arm systems

Slide 17 — Advantages of This System

- **No UI required** — mode switching is fully automatic from sensor context
- **Low-cost hardware** — entire system built on open-source tools and commodity components
- **Globally accessible** — remote control works from any phone browser anywhere in the world with internet
- **Modular architecture** — each ROS2 node is independent; any component can be replaced or upgraded
- **Simulation-first approach** — all software validated in Gazebo before touching real hardware
- **ESP8266 wireless bridge** — eliminates USB cable tether; future Wi-Fi serial possible
- **Safety-by-design** — multiple independent safety layers at firmware, controller, and planner levels
- **Open-source stack** — ROS2 + Gazebo + MoveIt2 + MediaPipe + YOLOv8; fully reproducible

---

Slide 18 — Challenges Encountered & Solutions

Challenge	Solution
SolidWorks CAD model — no software access	Used **Onshape browser CAD + API** to convert to STL without installing SolidWorks
40-link CAD URDF — too complex for simulation	Computed cumulative transforms via **Python/NumPy** to merge into clean 12-link model
Gazebo Harmonic — 9 separate simulation errors	Systematic debugging: inertia fix, URI paths, material strings, joint types, relay node
RViz2 dropping all joint state messages	Created custom **joint_state_relay.py** ROS2 node to stamp `frame_id` on bridge output
Gripper free-spinning after mimic joint removal	Changed to `revolute` type with explicit limits and `<dynamics>` damping
Coordinate frame mismatch in hand tracking	Normalising MediaPipe landmark coordinates relative to wrist anchor point

PROF

Slide 19 — Conclusion

What Was Built

This project is not a feature demonstration — it is a **complete autonomous robotic system** with an integrated human control layer:

System Component	What Was Achieved
Full robotics pipeline	Perception → Localisation → Planning → Execution — autonomous, end-to-end
Custom simulation environment	Gazebo Harmonic with real physics, 43-mesh robot, full TF tree — built from scratch
Production-grade URDF	Physical hardware → SolidWorks → Onshape → 40-link → optimised 12-link model
AI vision pipeline	YOLOv8 + OpenCV + camera calibration + pixel-to-world coordinate transform
Real-time gesture control	MediaPipe 21-landmark hand tracking → 6 joint angle mapping → live arm control
Global teleoperation	Phone camera → MQTT → ROS2 → arm — works from anywhere in the world
Unified architecture	Single mode manager handles all three modes with interrupt-safety guarantees

Why This Is Significant

- Solves a **real open problem** in industrial robotics: safe, intuitive, hands-free human override of an autonomous system
- Demonstrates that **industrial-grade autonomous robotics** is achievable with open-source tools and commodity hardware
- Produces a **reproducible, extensible research platform** — any component (vision, planner, controller) can be swapped or upgraded independently
- Replaced Arduino Mega with **ESP8266** to enable wireless hardware bridging — removing the physical tether entirely
- The autonomous sorting system and the human control layer are both **independently complete** engineering achievements combined into one unified system

PROF

Slide 20 — Future Scope

- **Complete gesture-to-arm mapping** — finalise landmark-to-joint-angle pipeline and test live arm mirroring in simulation
- **MoveIt2 integration** — run Setup Assistant, configure planning groups, implement IK-based motion execution

- **Autonomous sorting pipeline** — integrate YOLOv8 object detection and full pick-sort-return cycle in Gazebo
- **Remote MQTT control** — build phone web app, configure HiveMQ broker, test internet-controlled arm
- **Hardware deployment** — flash ESP8266 firmware, wire PCA9685, implement `ros2_control` hardware interface
- **Raspberry Pi migration** — move ROS2 stack to RPi 4/5 for a self-contained embedded system
- **Custom YOLOv8 training** — train on lab-specific objects using Roboflow for higher sorting accuracy
- **Bi-directional haptic feedback** — force sensing on gripper with feedback to human operator during remote control
- **Multi-arm coordination** — extend architecture to coordinate two arms for assembly tasks

---

Slide 21 — Thank You

Thank You

Questions & Discussion

Slide Count Summary

#	Section	Purpose
1	Title	—
2	Abstract	Frames the full system scope
3	Introduction	Industry context + market justification
4	Problem Statement	3 distinct engineering problems
5	System Overview	Autonomous as primary; gesture as innovation
6	**Autonomous Sorting Pipeline**	Technical depth of the core system
7	Hardware	Components including ESP8266 rationale
8	Software Stack	Full technology table
9	ROS2 Architecture	Node graph
10	Remote Control Flow	MQTT data path
11	URDF & Simulation	CAD → simulation journey
12	MediaPipe Integration	Gesture pipeline
13	Progress & Roadmap	Current status
14	Flowchart	System operation logic
15	Safety Features	Reliability guarantees

#	Section	Purpose
16	Applications	Industry use cases
17	Advantages	Why this approach
18	Challenges & Solutions	Real technical problems solved
19	Conclusion	Full scope summary
20	Future Scope	Roadmap forward
21	Thank You	—
Total		**21 slides**